Design Proposal: Extensible Vocabulary-to-Glossary Mapping (PEO Semantic Service)

Nov 6, 2026 · @Jeffrey Wallk

Summary and decision requested

We propose linking core and domain vocabulary terms to PEO glossary entries and acronyms through separate, versioned named graphs, so a new glossary is added without changing anything that exists.

Each glossary, vocabulary and mapping set lives in its own graph, loaded through the INBOUND Semantic Service facade's pre-defined ingestion pipe. Terms and acronyms are SKOS-XL labels with their own URIs. Links between them are SKOS match relations, recorded as reified mapping records that carry status, scope and provenance.

Decision requested: approve the graph layout, information model and URI strategy, and fund phase 1 of the roadmap with one core vocabulary, one domain vocabulary and one glossary as the pilot.

How to reuse this document: everything through Consumption patterns is the architecture and stays fixed. The Extension playbook and the Appendix are the template: copy them to propose each new glossary, vocabulary or mapping set.

Context and problem

The PEO maintains many glossaries and acronym sets, and each is interpreted differently by the core and domain vocabularies that use them.

Today's pain points (confirm and quantify before approval):

- The same acronym (for example PTO, ACA, FMLA) means different things by context, and nothing records which meaning applies where.
- Links between vocabulary terms and glossary entries, where they exist, are embedded in the source, so a glossary reload or a core change forces coordinated change.
- Nobody can answer why two terms are linked, who approved the link, or whether it is still valid.

- Each new glossary repeats the integration work, because there is no standard way to onboard one.

Scope: core vocabularies, domain vocabularies (payroll, benefits, HR and others), glossaries and acronym sets, and the mappings between them, all served through the INBOUND Semantic Service facade.

Out of scope: authoring tools for glossaries, natural-language search UX, and changes to source-system data models.

Goals, non-goals and success criteria

The design succeeds if a new glossary can be added without editing any existing graph, and if every link can explain itself.

Goal	Success criterion	How measured
Reuse	One glossary term is linked from many vocabularies without copying	Share of glossary terms with 2 or more inbound mappings
Isolation from change	Reloading any one graph requires no edits to another	Zero cross-graph write dependencies; reload needs no coordinated release
Understanding	Any mapping answers who, why, when and in what context	100% of Accepted mappings carry status, justification, scope and provenance
Extensibility	Onboarding a new source follows one repeatable procedure	Median onboarding effort in days, tracked per source [target: to be set after pilot]
Trust	Broken or stale links are found automatically	Dangling-reference report produced on every load

Non-goals:

- Merging vocabularies or glossaries into one canonical model.
- Resolving acronyms globally. Every acronym mapping is scoped.
- Replacing the source systems of record for any term.

Design principles and constraints

Five rules drive every later decision, and a proposal that breaks one needs an explicit exception.

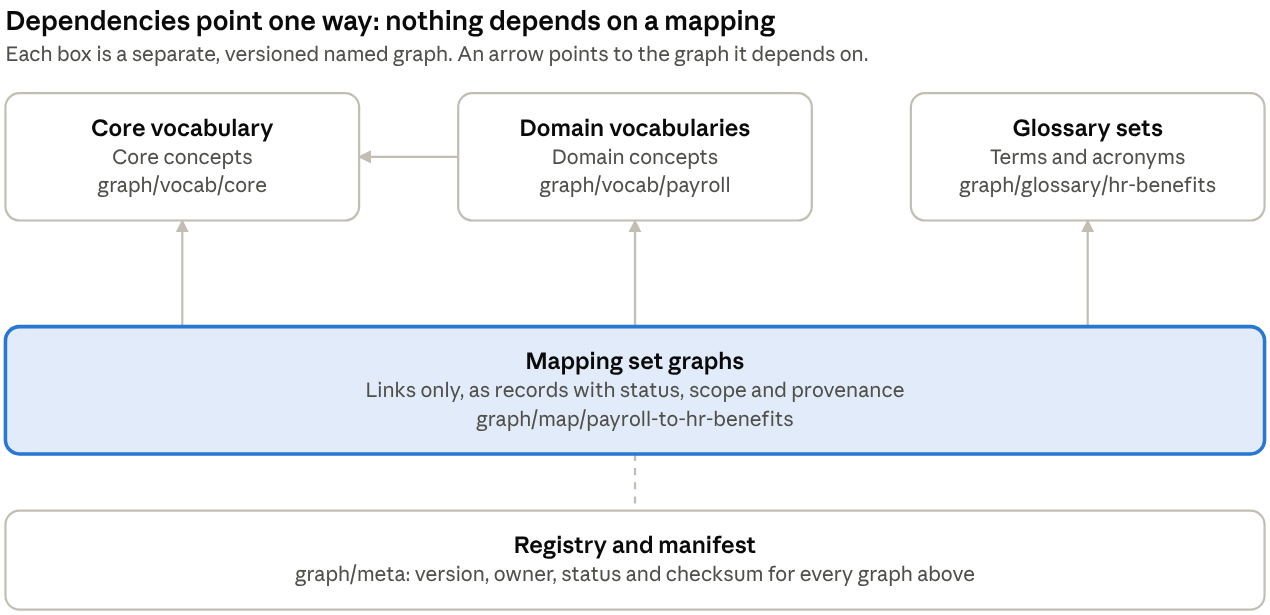
1. **One-way dependencies.** Domain depends on core, mappings depend on the graphs they connect, and nothing depends on a mapping.
2. **Single ownership.** A glossary owns its definitions, a vocabulary owns its concepts, and a mapping owns only the relationship.
3. **Version the graph, not the term.** Term URIs are stable across versions. Graphs are immutable once published.
4. **Weakest true predicate.** Prefer closeMatch, broadMatch or relatedMatch over exactMatch. Never use owl:sameAs across sources.
5. **Context-scoped acronyms.** An acronym never resolves globally. It resolves within a stated domain or scheme.

Constraints:

- All loads go through the INBOUND Semantic Service facade's pre-defined ingestion pipe. No side-channel writes.
- Loads are idempotent and replace by graph.
- Standards first: SKOS, SKOS-XL, SHACL, DCAT/VoID, PROV-O, with an SSSOM-style mapping record.
- Platform-specific graph limits, named-graph support and SHACL engine choice are [to be confirmed with the platform team].

Architecture

Five kinds of named graph carry the design, and every dependency runs in one direction.



graph layout · 5 graph types, one-way dependencies

Domain vocabularies depend on core, mapping sets depend on the graphs they connect, and the registry describes all of them.

- Never write mapping triples into a source graph. A glossary reload would then become a coordinated change.
- Publish each graph as an immutable version. The registry says which version is current.
- Consumers select graphs at query time (see Consumption patterns).

Information model

Concepts are SKOS, terms and acronyms are SKOS-XL labels with their own URIs, and each link is a mapping record rather than a bare triple.

Glossary entries as lexical resources. This lets an acronym carry its own source, status and context, and makes the expansion an explicit relation instead of string parsing.

```
# Glossary graph
g:term-FMLA a skosxl:Label ;
    skosxl:literalForm "FMLA"@en ;
    ex:isAcronymOf g:term-FamilyMedicalLeaveAct ;
    dct:source g:src-hr-benefits ;
    ex:status ex:Approved .

g:term-FamilyMedicalLeaveAct a skosxl:Label ;
    skosxl:literalForm "Family and Medical Leave Act"@en ;
    skos:definition "..."@en .

# Vocabulary graph (core or domain)
v:LeaveOfAbsence a skos:Concept ;
    skosxl:prefLabel v:label-LeaveOfAbsence ;
    skos:inScheme v:CoreScheme .
```

Mapping predicates. Use the weakest one that is true.

Predicate	Use when	Caution
skos:exactMatch	Interchangeable in any context	Transitive, so errors cascade. Use sparingly and review chains
skos:closeMatch	Interchangeable in some contexts	Default for cross-domain links
skos:broadMatch / narrowMatch	Hierarchical alignment	Check direction
skos:relatedMatch	Associative link	Not for definitions

Mapping record. Anything beyond a bare link is modelled as a record, in the style of SSSOM.

```
m:0001 a ex:Mapping ;
    ex:subject     payroll:LeaveAccrual ;
    ex:predicate   skos:closeMatch ;
    ex:object      g:term-FMLA ;
    ex:confidence  0.86 ;
    ex:justification ex:LexicalMatch ;   # or ManualReview,
EmbeddingSimilarity
    ex:status      ex:Proposed ;         # Proposed -> Accepted -> Deprecated
    ex:domainScope payroll:Scheme ;      # disambiguates acronyms
    prov:wasAttributedTo <user/or/pipeline> ;
    dct:created    "2026-11-06"^^xsd:date .
```

The `ex:` terms are a small mapping ontology that this project would publish and version as its own graph.

URI and versioning strategy

Term URIs are permanent and opaque. Graph URIs carry the version. This is what lets a reload replace a graph without breaking any link to a term.

Item	Pattern	Rule
Term or concept	`https://{host}/id/{source}/{stableId}`	Opaque ID. Never embed the label, since labels change
Namespace	One per source graph	Ownership is visible from the URI
Graph	`https://{host}/graph/{type}/{source}/v{n}`	Immutable once published. New content is a new version
Mapping record	`https://{host}/id/map/{mapSet}/{stableId}`	Owned by the mapping set that created it
Registry pointer	`https://{host}/graph/meta`	Records which version is current per source

Lifecycle rules:

- Concepts keep their URI across graph versions.

- Retire with `owl:deprecated true` and `dct:isReplacedBy`. Never delete.
- Mappings that point to a deprecated target are found by query and migrated through a new mapping version.
- A breaking change to a source is a new major graph version, and consumers move when ready.

The host name, stable ID format (UUID, sequence or hash) and who mints IDs are [decisions needed], listed under Open questions.

Validation and governance

Every graph type has a SHACL shape, and every mapping moves through a defined lifecycle with a named steward.

Validation gates (a load fails if any gate fails):

- Every concept, label and mapping has the properties its shape requires.
- Every mapping subject and object resolves in a registered graph.
- Every acronym mapping has a domain scope.
- No exactMatch cycle or chain longer than the agreed limit [limit to be set].
- Registry metadata (source, version, checksum, load time) is present.

Mapping lifecycle:

Status	Meaning	Who moves it
Proposed	Created by a person or by candidate generation	Pipeline or author
Accepted	Reviewed and approved for use	Domain steward
Deprecated	No longer valid, with a replacement if one exists	Domain steward

Roles:

Role	Responsibility
Glossary owner	Definitions and acronym expansions in their graph
Vocabulary owner	Concepts and schemes in their graph
Mapping steward	Reviews and accepts mappings for a domain pair

Role	Responsibility
Platform owner	Pipeline, shapes, registry and the mapping ontology

Change control: a change to a source graph publishes a new version and a dangling-reference report. Mapping stewards review affected mappings. Consumers are never forced to move.

Consumption patterns

Consumers compose the graphs they need at query time and never receive a merged copy.

```
SELECT ?concept ?term ?def
FROM <.../graph/vocab/core/v3>
FROM <.../graph/vocab/payroll/v2>
FROM <.../graph/glossary/hr-benefits/v5>
FROM <.../graph/map/payroll-to-hr-benefits/v1>
WHERE {
  ?m ex:subject ?concept ; ex:object ?term ; ex:status ex:Accepted .
  OPTIONAL { ?term skos:definition ?def }
}
```

Rules for consumers:

- Filter on `ex:status ex:Accepted` by default. Proposed mappings are for stewards only.
- Pin graph versions in the query, or resolve current versions through the registry.
- For acronyms, always include the domain scope in the pattern.

Derived index graph. For fast lookup, the pipeline may build a concept-to-term-to-acronym index graph. It is disposable and rebuilt from sources on every load. It is never edited and never a source of truth.

Facade contract. The INBOUND Semantic Service facade should expose lookups by concept, by term and by acronym with scope, so consumers do not hand-write graph lists. The facade's exact operations are [to be defined with the service team].

Extension playbook

Adding a source is always the same three artifacts: a graph, a manifest entry, and a mapping set. Nothing existing is edited.

You are adding	You create	You touch
A glossary or acronym set	Glossary graph, manifest entry, mapping set to core or domain terms	Nothing existing
A domain vocabulary	Vocabulary graph, manifest entry, mappings to core (and to glossaries where core has no home)	Nothing existing
A new version of a source	New graph version, updated registry pointer, dangling-reference report	Registry pointer only
Mappings between two existing graphs	A mapping set graph	Nothing existing

Onboarding procedure (copy the checklist from the Appendix for each source):

1. **Propose.** Complete the source onboarding template: owner, scope, licence, expected volume, refresh cadence.
2. **Register.** Create the manifest entry and reserve a namespace.
3. **Transform.** Convert the source to SKOS or SKOS-XL with stable IDs.
4. **Validate.** Run the SHACL shapes for its graph type.
5. **Load.** Run the ingestion pipe into a new versioned graph.
6. **Map.** Link to core first, then to other domains only where core has no home. Candidates land as Proposed.
7. **Review.** The mapping steward accepts, rejects or rescopes.
8. **Publish.** Flip the registry pointer and announce the version.

Mapping order of preference: source to core, then source to source only when core has no concept. This keeps mapping count roughly linear in the number of sources instead of N times M.

Risks and mitigations

The largest risks are mapping sprawl and acronym ambiguity, and both are controlled by structure rather than by review effort.

Risk	Effect	Mitigation
Point-to-point mapping explosion (N domains by M glossaries)	Mapping count and review load grow multiplicatively	Map to core first. Direct links only where core has no home
Acronym collisions	Wrong expansion shown in the wrong context	Mandatory domain scope on acronym mappings. No global resolution
Transitive exactMatch chains	One bad link corrupts a cluster	Prefer closeMatch. Cap and review chains. Shape check on exactMatch
Ownership blur	A glossary reload overwrites mappings or definitions	Mappings in their own graphs. Source graphs never receive mapping triples
Stale mappings after a source changes	Links point to deprecated or missing terms	Dangling-reference report on every load. Deprecate, never delete
Low-quality automated candidates	Steward review queue floods	Candidates stay Proposed. Confidence threshold. Sampled review
Platform limits on named graph count or size	Load or query degradation	Validate in the pilot. Archive old versions. Use the derived index for hot paths

Alternatives considered

We rejected four alternatives because each couples sources to each other, which is the problem this design removes.

Alternative	Appeal	Why rejected
Merge all sources into one graph with owl:sameAs	Simple queries	Merges identity. A meaning that drifts in one domain corrupts all. A reload becomes a coordinated change
Embed links inside each source graph	Fewer graphs	Source reload overwrites links. Ownership unclear. Links cannot carry their own lifecycle
String or label matching at query time	No modelling effort	Fails on acronym ambiguity. No provenance. Not auditable
Plain SKOS match triples with no mapping record	Lightest model	Cannot answer who, why, when or in what context. Acceptable only for low-stakes links, and can be derived from records

Open questions and decisions needed

Six decisions block or shape phase 1, and none has an owner yet.

- [ ] Host name and stable ID format for URIs, and who mints IDs
- [ ] Platform limits: named graph count and size, SHACL engine, and support for graph-level versioning
- [ ] Which core vocabulary is the first mapping target, and who owns it
- [ ] Pilot scope: which domain vocabulary and which glossary go first
- [ ] Exact-match policy: maximum chain length, and who approves exactMatch
- [ ] Facade operations to expose for lookup by concept, term and scoped acronym

Appendix: reusable templates

Copy these for each new source or mapping set. Replace every value in braces.

A. Source onboarding proposal (one page per source)

```
# Onboarding: {source name}

- Type: {glossary | acronym set | core vocabulary | domain vocabulary |
mapping set}
- Owner / steward: {name, team}
- Purpose and consumers: {who uses it, for what}
- Scope and licence: {coverage, licence, sensitivity}
- Expected volume: {terms, acronyms, mappings}
- Refresh cadence: {event-driven | weekly | quarterly}
- Depends on: {core graph(s), or none}
- Namespace and graph URI: {https://{host}/id/{source}/ ,
.../graph/{type}/{source}/v1}
- Mapping targets: {core concepts first, then other sources only where core
has no home}
- Acronym scopes needed: {domain schemes}
- Risks and exceptions to the design principles: {none | list}
- Approvals: {owner, mapping steward, platform owner}
```

B. Registry manifest entry

```
<https://{host}/graph/meta/{source}/v1> a dcat:Dataset, void:Dataset ;
    dct:title        "{source name}" ;
    dct:publisher    {owner} ;
    dct:hasVersion   "{n}" ;
    dct:issued       "{date}"^^xsd:date ;
    ex:graphType     ex:{GlossaryGraph | VocabularyGraph | MappingGraph} ;
    ex:dependsOn     <{graph URI}> ;
    ex:status        ex:{Draft | Current | Superseded} ;
    ex:checksum      "{sha256}" .
```

C. SHACL shape for a mapping record (starting point)

```
ex:MappingShape a sh:NodeShape ;
    sh:targetClass ex:Mapping ;
    sh:property [ sh:path ex:subject    ; sh:minCount 1 ; sh:maxCount 1 ;
sh:nodeKind sh:IRI ] ;
    sh:property [ sh:path ex:object     ; sh:minCount 1 ; sh:maxCount 1 ;
sh:nodeKind sh:IRI ] ;
    sh:property [ sh:path ex:predicate  ; sh:minCount 1 ; sh:maxCount 1 ;
                  sh:in ( skos:exactMatch skos:closeMatch skos:broadMatch
                          skos:narrowMatch skos:relatedMatch ) ] ;
    sh:property [ sh:path ex:status     ; sh:minCount 1 ; sh:maxCount 1 ;
                  sh:in ( ex:Proposed ex:Accepted ex:Deprecated ) ] ;
    sh:property [ sh:path ex:justification ; sh:minCount 1 ] ;
    sh:property [ sh:path prov:wasAttributedTo ; sh:minCount 1 ] ;
    sh:property [ sh:path dct:created   ; sh:minCount 1 ; sh:datatype
xsd:date ] .
```

Extra rules to add as SPARQL constraints: acronym objects require `ex:domainScope`, and subject and object must resolve in a registered graph.

D. Pre-load and post-load checklist

- [ ] Manifest entry created and namespace reserved
- [ ] Source converted with stable IDs and no label-based URIs
- [ ] SHACL validation passes for the graph type
- [ ] Loaded into a new versioned graph, never over an existing one
- [ ] Dangling-reference report reviewed
- [ ] Candidate mappings created as Proposed only
- [ ] Mapping steward review complete
- [ ] Registry pointer flipped and consumers notified